InnovatEPAM Portal

A201 “Beyond Vibe Coding” — Capstone Showcase

A reference build for AI-native engineering:
Spec-Driven Development · Context Engineering · Constitution-as-CI

Fatih Furkan Bilsel

12 May 2026

GitHub Copilot + SpecKit

What this deck demonstrates

- **This is a bootcamp capstone**, not a product pitch — the artefact is the *process*, the app is the proof.
- **AI as a force multiplier, not the architect.** Every Copilot output is constrained by spec, ADR, types, tests and a CI "constitution".
- **Engineering judgement is visible** in the file tree: 2 SpecKit features, 12 ADRs, a typed error registry, a pure-function state machine, custom CI checks.
- **Reproducible workflow**: `/speckit.specify → plan → tasks → implement` on every slice, same loop both phases.

What follows: the AI-native workflow I used, then a screen tour mapped to the technical decisions behind each surface.

My AI-Native Workflow

Context engineering

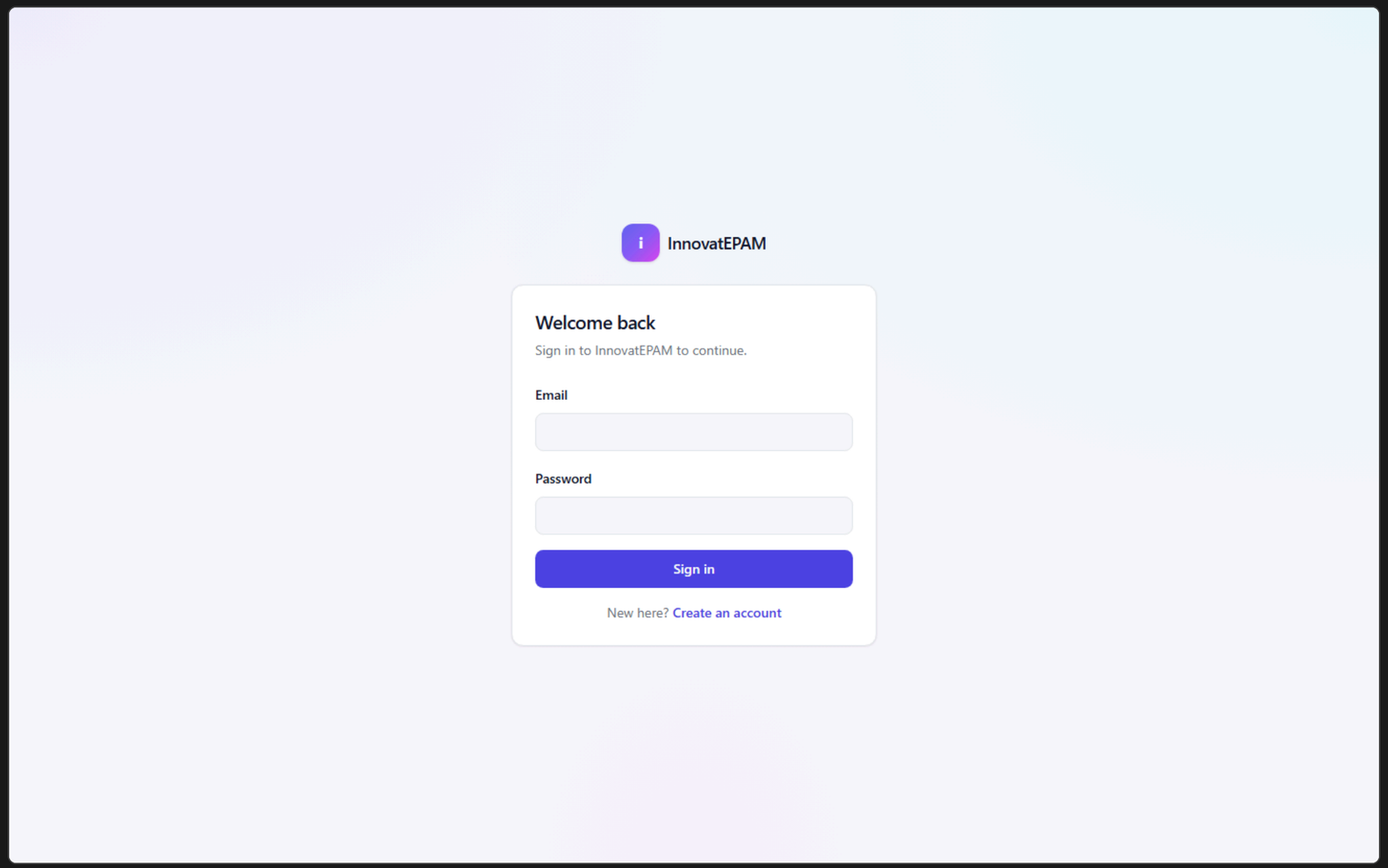
- `.github/copilot-instructions.md` = project constitution loaded into every Copilot turn
- Per-area `AGENTS.md` conventions (server, db, ui)
- Spec + ADRs pinned as context before code generation
- Custom checks (`check:error-codes`, `check:ui-tokens`) feed Copilot back its own mistakes

Prompt loop per slice

1. `/speckit.specify` — user stories, acceptance criteria
2. `/speckit.plan` — tech approach, data model, contracts
3. `/speckit.tasks` — atomic, testable units
4. Tests first, then implementation under Copilot
5. ADR for any irreversible call

Result: Copilot generates inside guardrails I designed — not the other way round.

Sign in (NextAuth + Argon2)



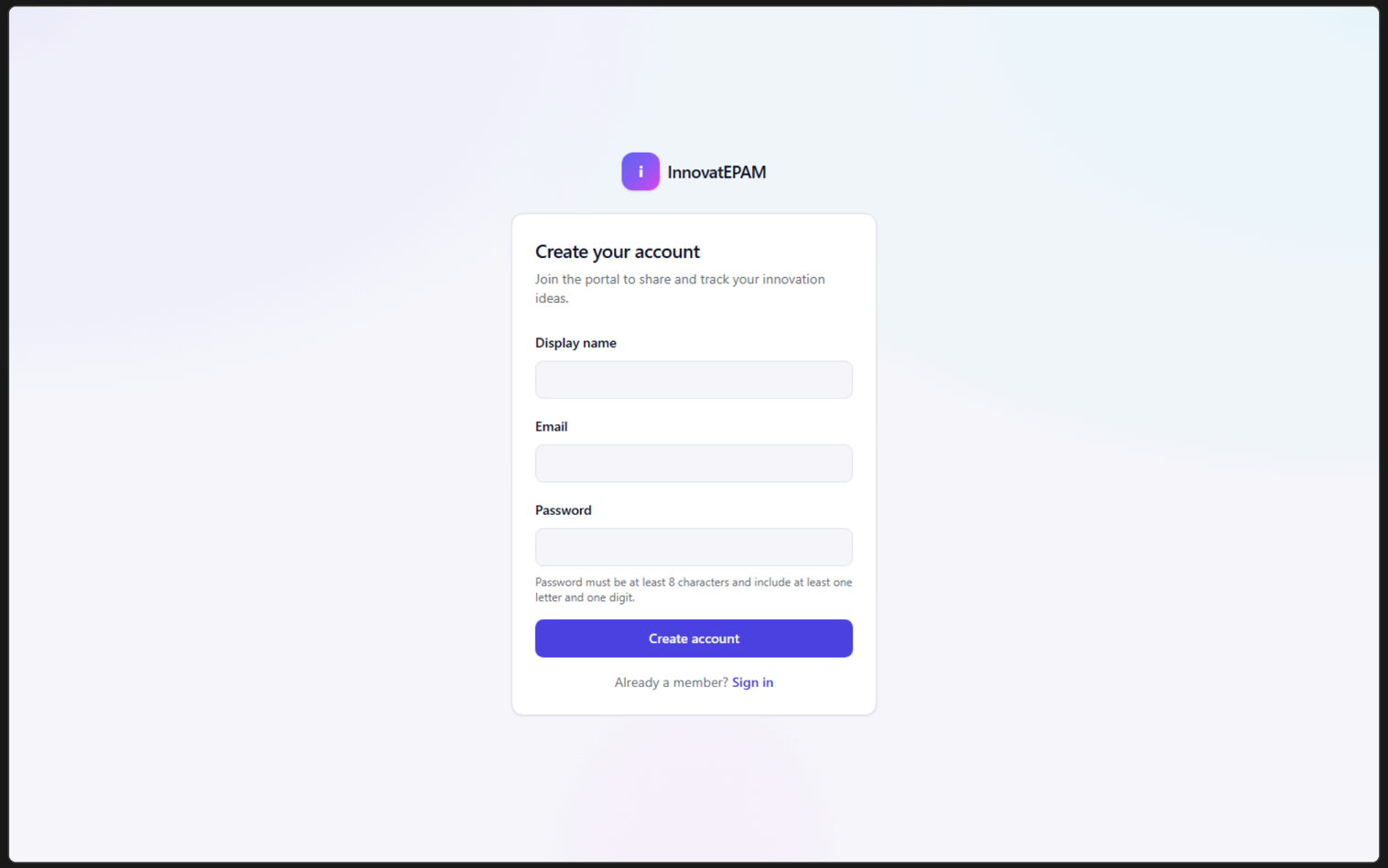
- Credentials provider, JWT session, 24 h sliding
- Argon2id password hashing
- Edge middleware redirects to `/login?callbackUrl=…`
- First admin elevated by one-shot `BOOTSTRAP_ADMIN_EMAIL` marker (audited)

FR-027 sliding expiry

audited login_success / login_failure

screenshots/01-login.png

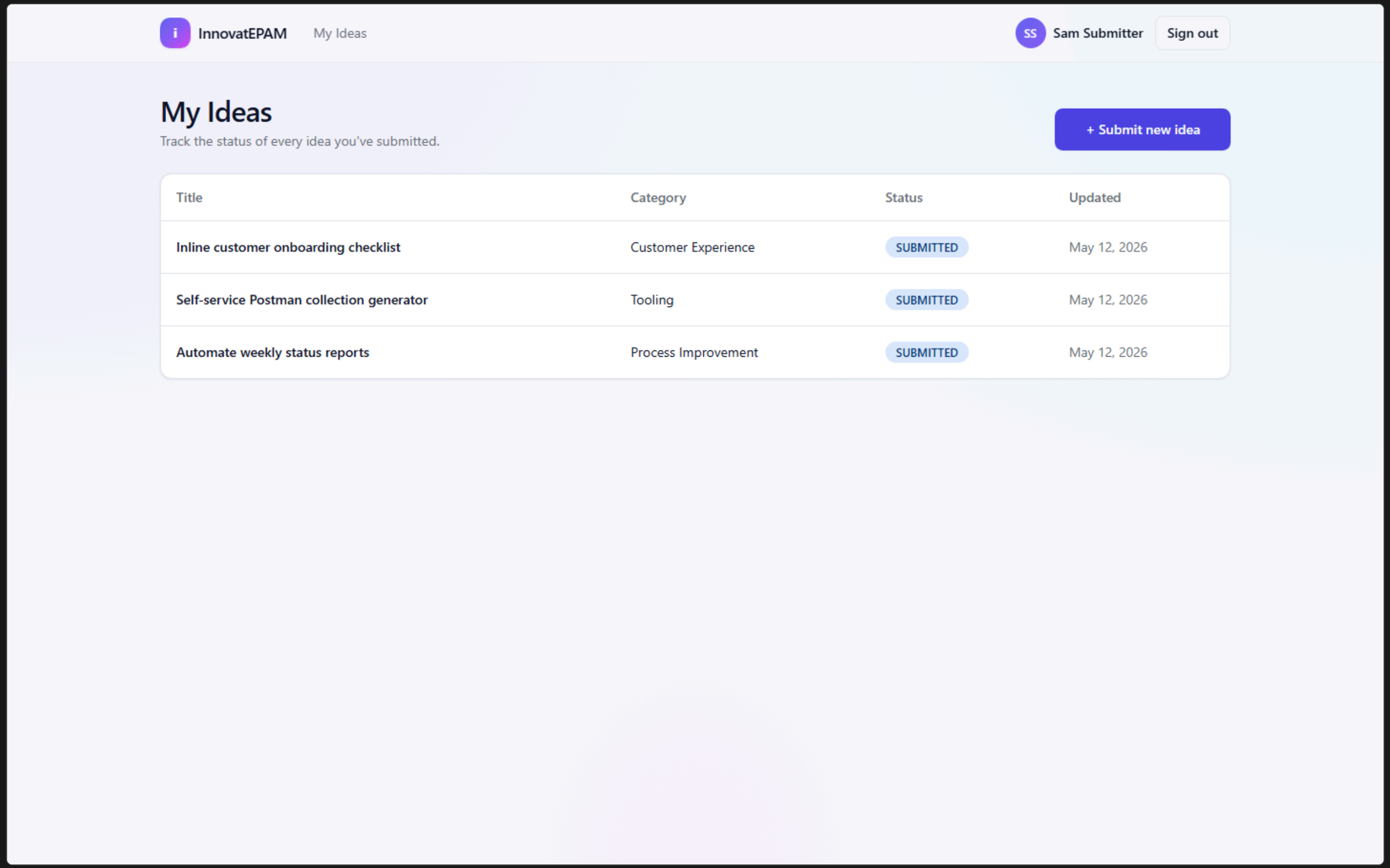
Register



- Self-service registration → starts as `EMPLOYEE`
- Password policy: ≥8 chars, ≥1 letter + ≥1 digit
- Server-side Zod validation, typed `ERROR_CODES`
- Rate-limited; emits `register` security event

screenshots/02-register.png

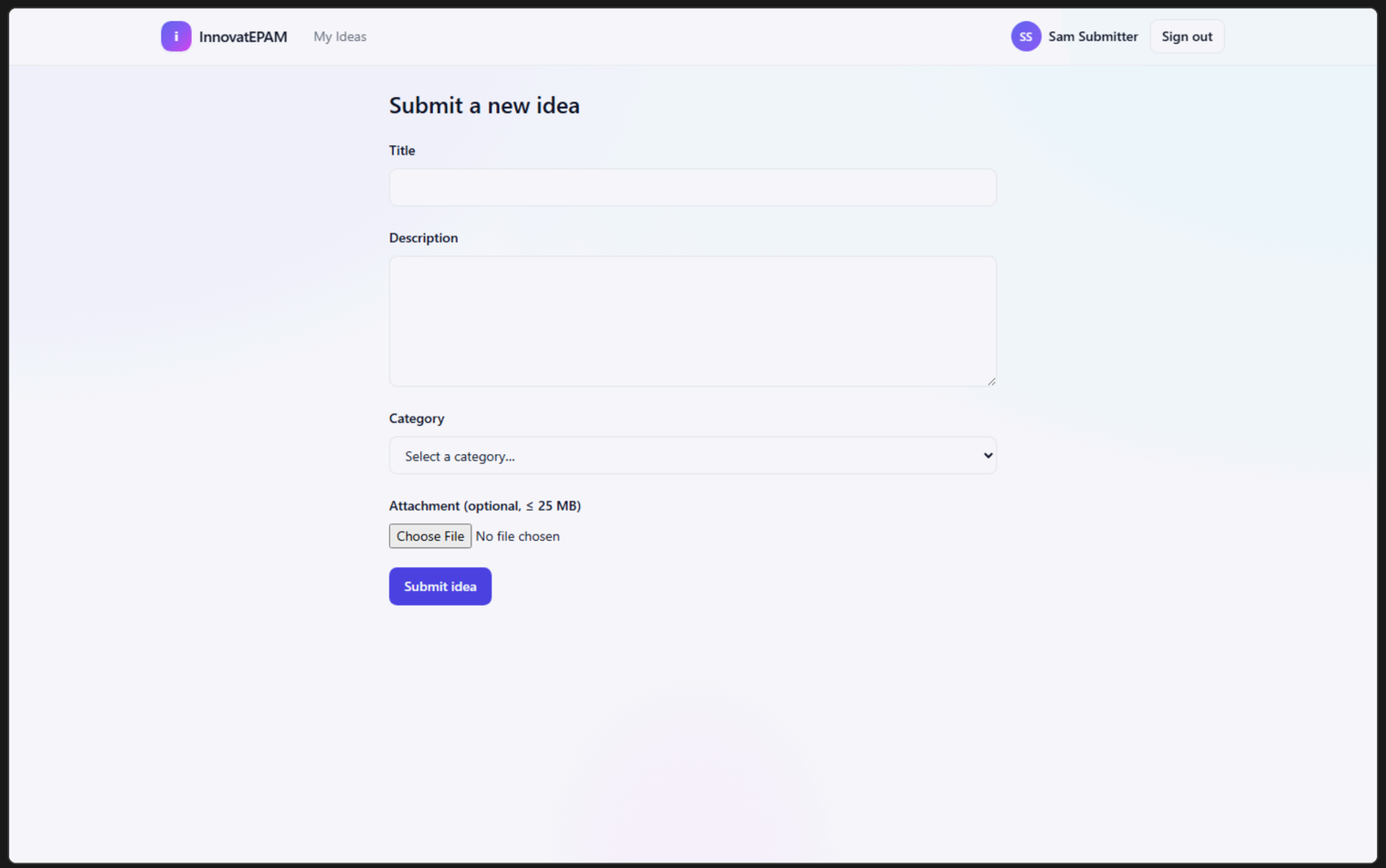
Employee — My Ideas



All ideas authored by the signed-in user, newest first, with status badges and category.

screenshots/03-employee-my-ideas.png

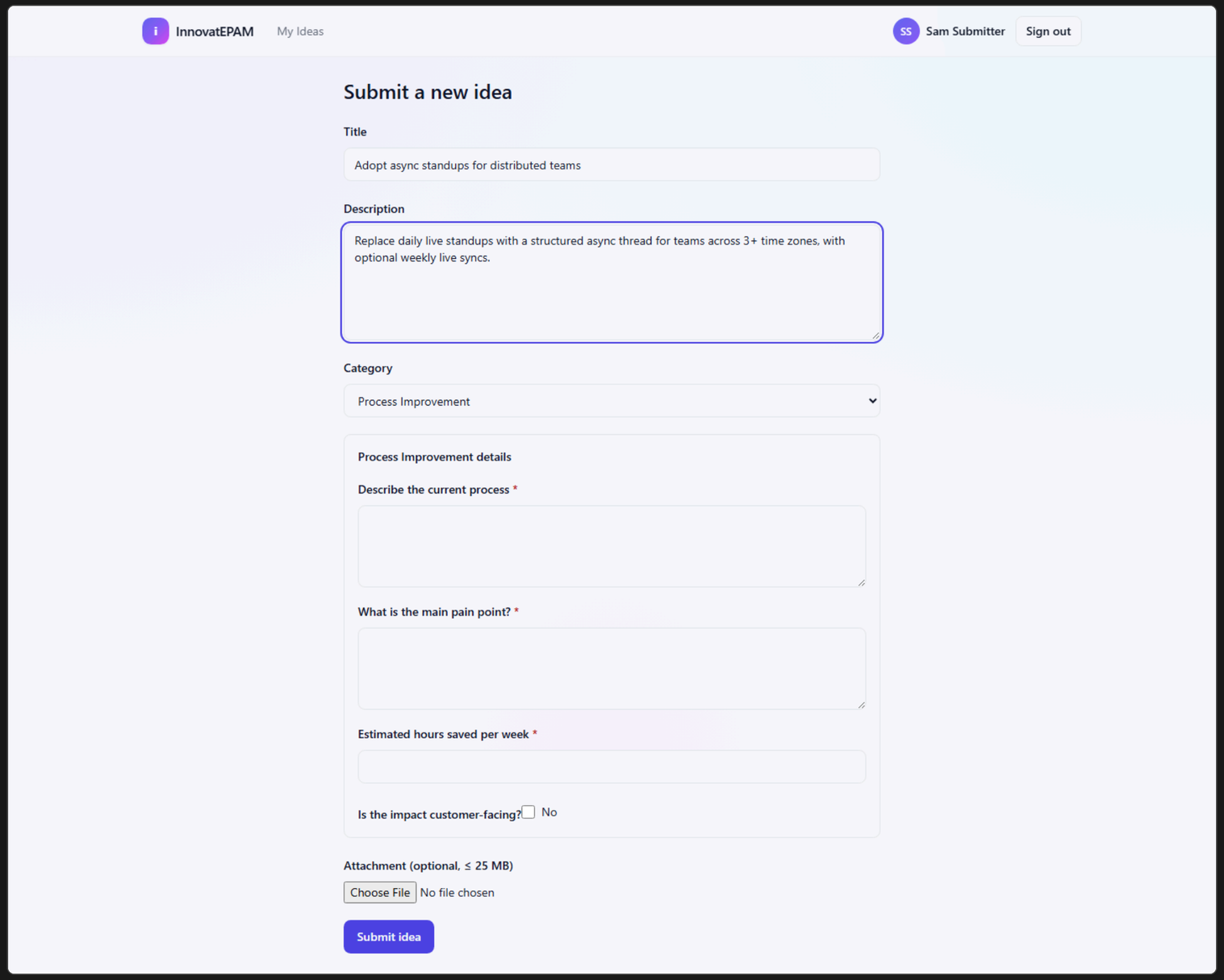
Submit a New Idea



- Title, description, category, optional attachment
- **One attachment** staged then committed atomically
- PDF / PNG / JPEG / DOCX / PPTX, ≤25 MB
- Magic-number MIME sniff (defence-in-depth)
- Auto-saved draft in `localStorage`

screenshots/04-employee-new-idea-empty.png

Smart Forms — Phase 2

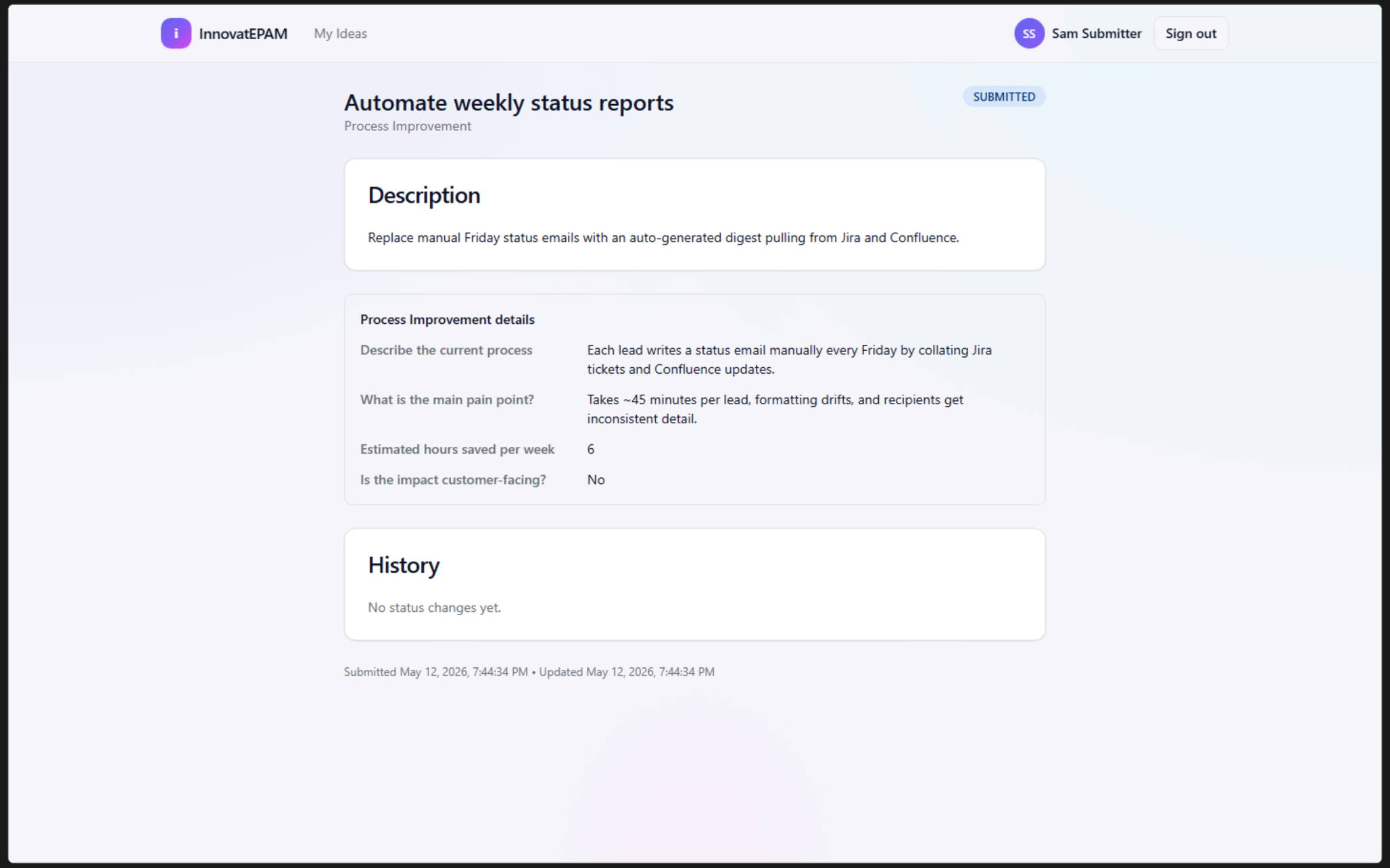


- Pick "Process Improvement" → **schema-driven fields appear inline**
- Field types: short text, long text, number, single-choice, yes/no
- **Runtime Zod schema** built from the live category definition
- Answers persist per-idea with **label snapshots** — review-time text survives later schema edits

ADR-0009 schema storage
ADR-0010 label snapshots
ADR-0011 dynamic Zod

screenshots/05-employee-new-idea-dynamic-fields.png

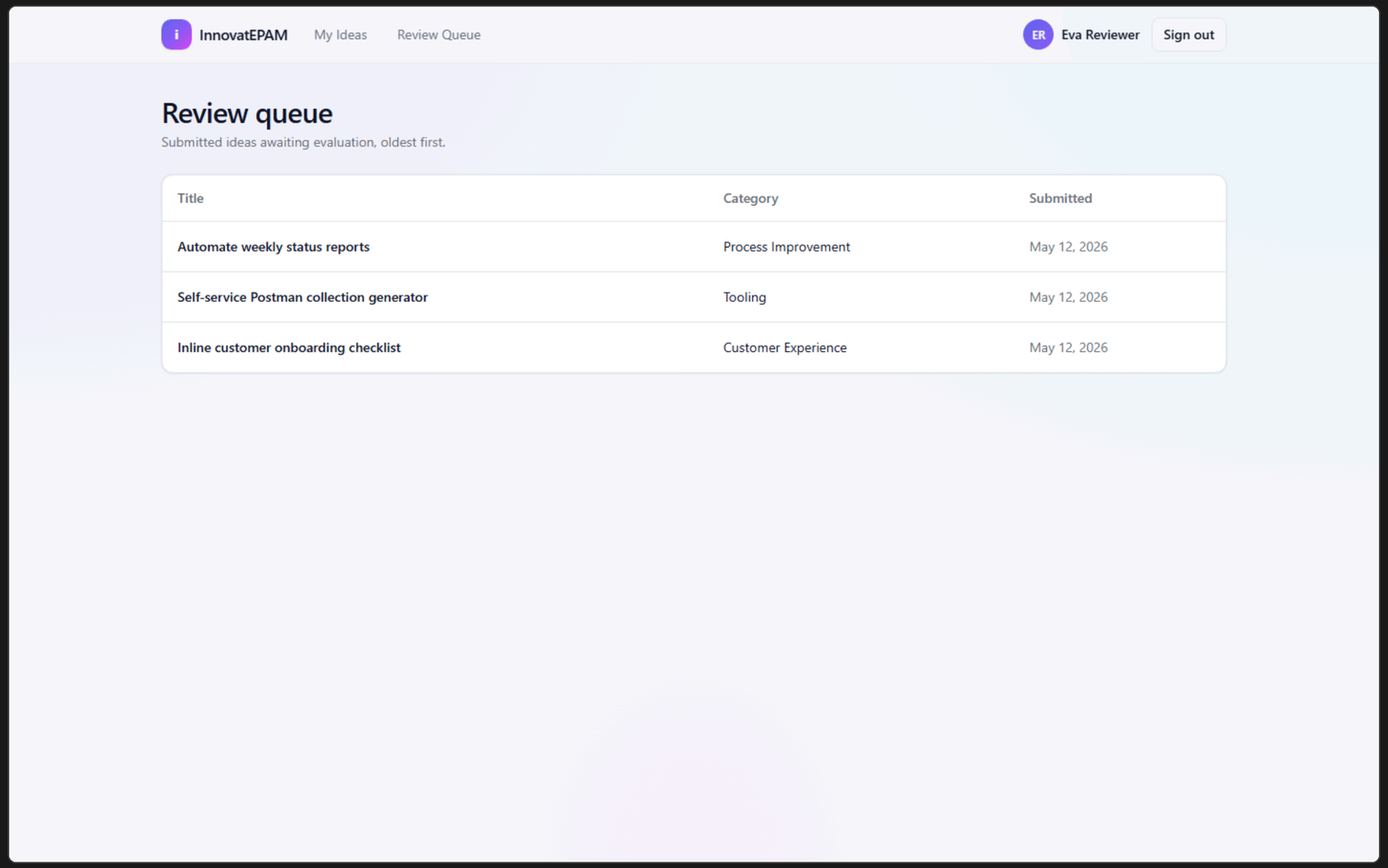
Idea Detail



Reads like a one-pager: status, description, attachment download, and the structured answers grouped under the category — rendered from the snapshot, not the live schema.

screenshots/06-employee-idea-detail.png

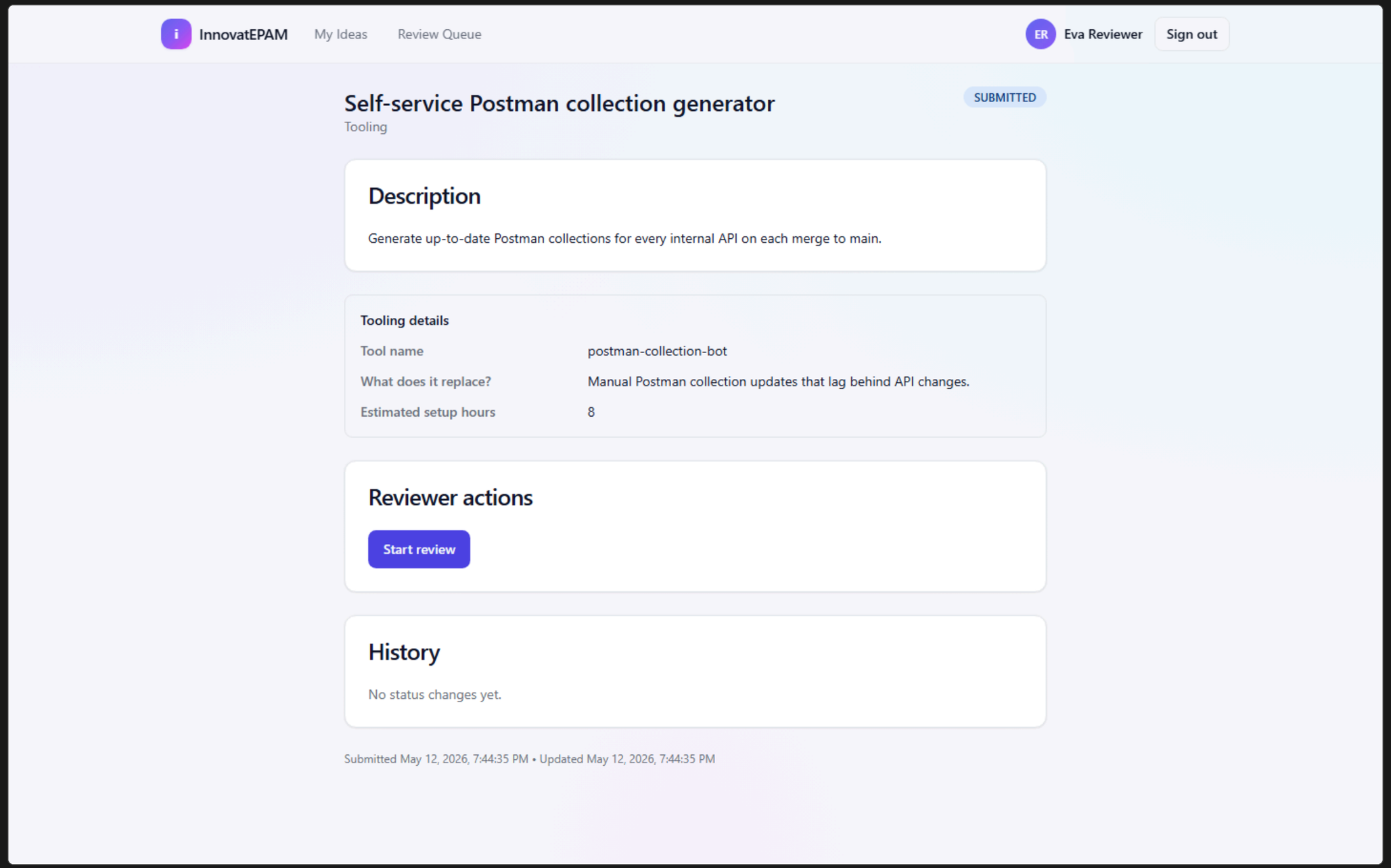
Reviewer — Queue



- All `SUBMITTED` ideas, oldest first
- Pending-category badge prevents premature decisions (FR-018)
- Self-evaluation forbidden — guarded both client- and server-side
- Role gate: `EVALUATOR` + `ADMIN` only

screenshots/07-reviewer-queue.png

Reviewer — Decide



Start review, **Approve**, **Reject** — comment required on approve/reject. Buttons are gated by the same pure-function state machine that the API enforces in the DB transaction.

screenshots/08-reviewer-idea-detail.png

Idea Lifecycle

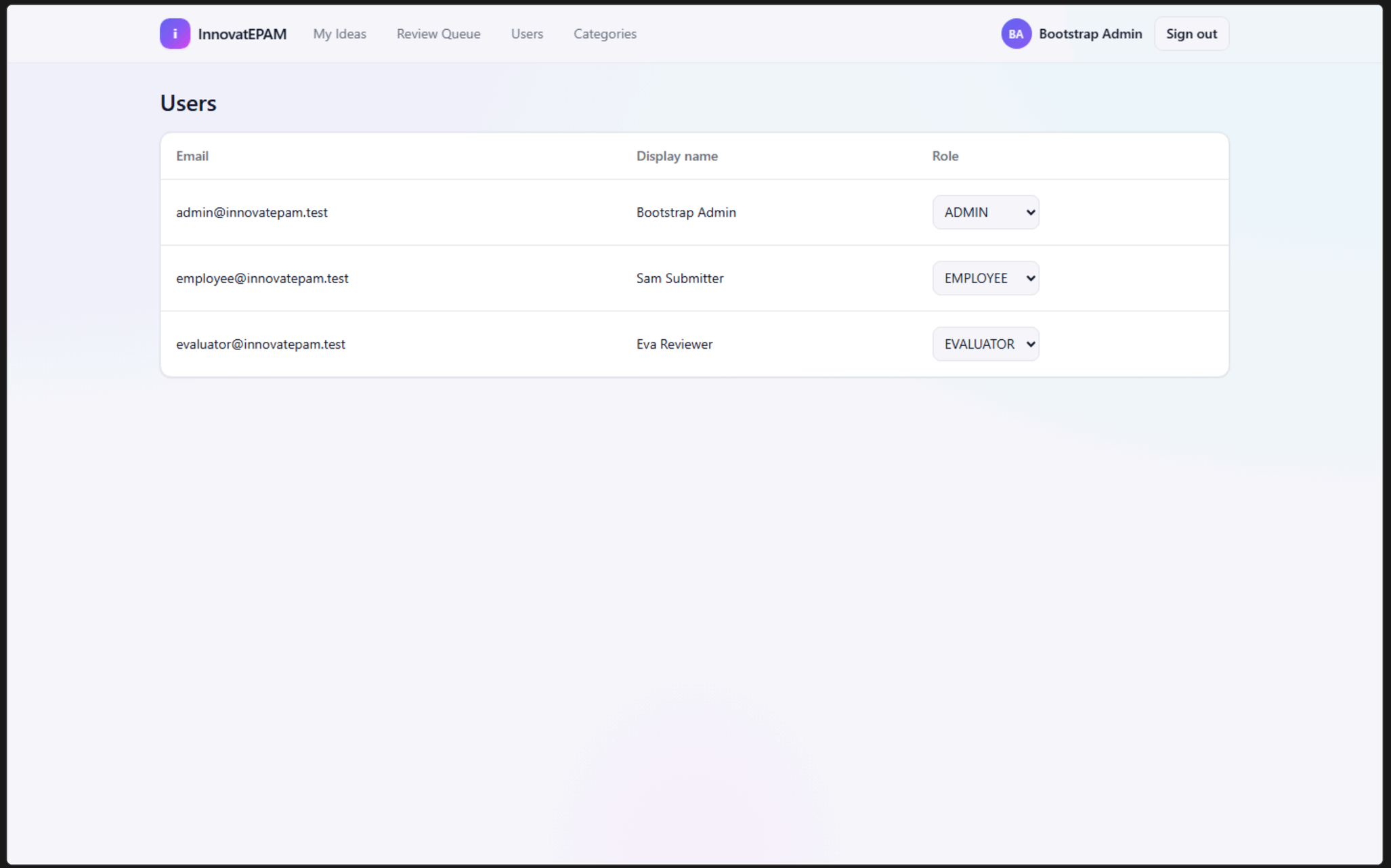
`SUBMITTED` → `UNDER_REVIEW` → `APPROVED` → `IMPLEMENTED`

REJECTED branches off from SUBMITTED or UNDER_REVIEW.

Action	From	To	Roles	Comment?
START_REVIEW	SUBMITTED	UNDER_REVIEW	EVALUATOR, ADMIN	—
APPROVE	SUBMITTED, UNDER_REVIEW	APPROVED	EVALUATOR, ADMIN	**required**
REJECT	SUBMITTED, UNDER_REVIEW	REJECTED	EVALUATOR, ADMIN	**required**
IMPLEMENT	APPROVED	IMPLEMENTED	ADMIN	—

Pure function. Same evaluator runs in the API transaction *and* in the UI for button gating. Every transition appended to the `status_transitions` audit table.

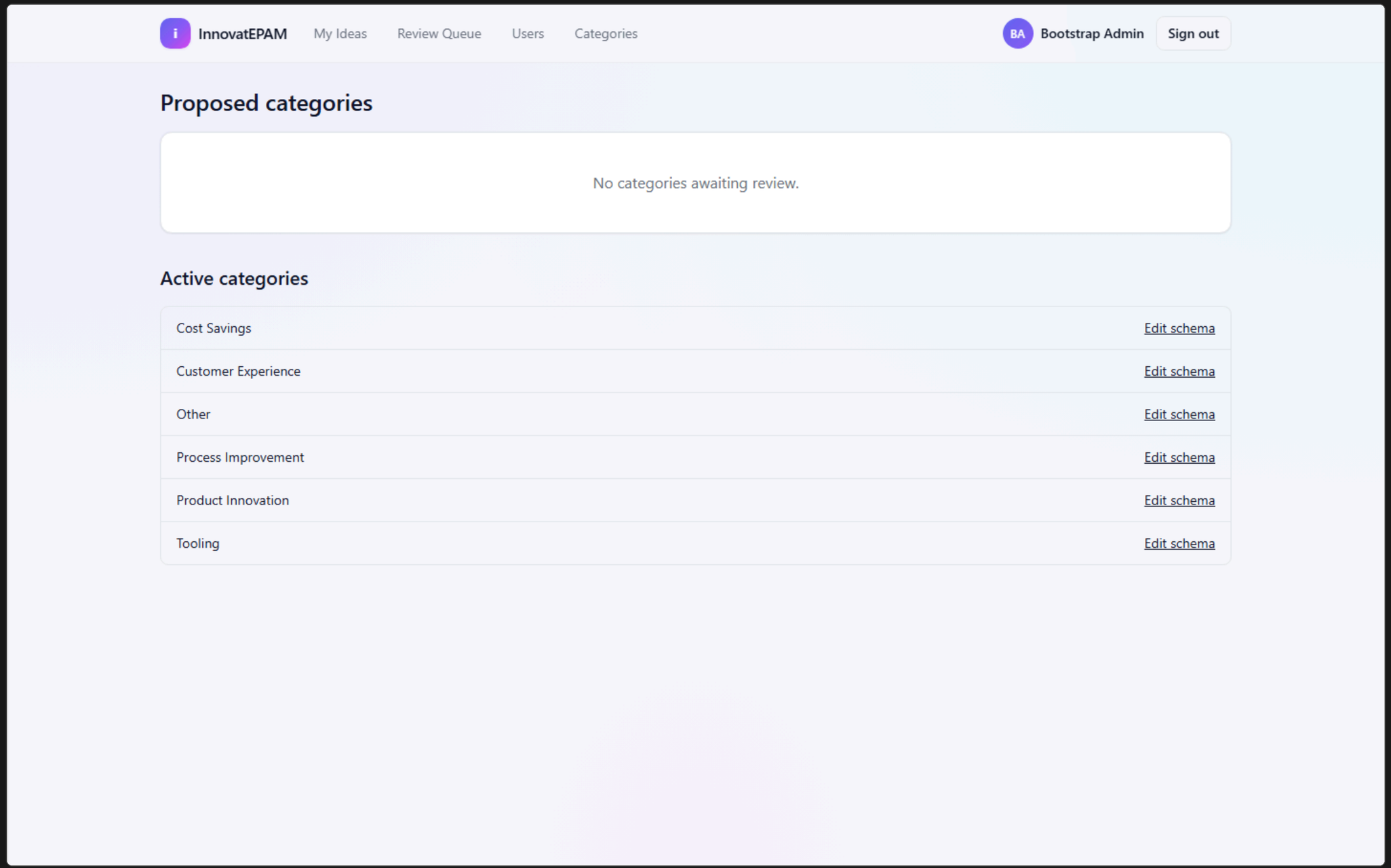
Admin — Users



Promote / demote between EMPLOYEE / EVALUATOR / ADMIN. Self-demotion is blocked. Every role change is audited.

screenshots/09-admin-users.png

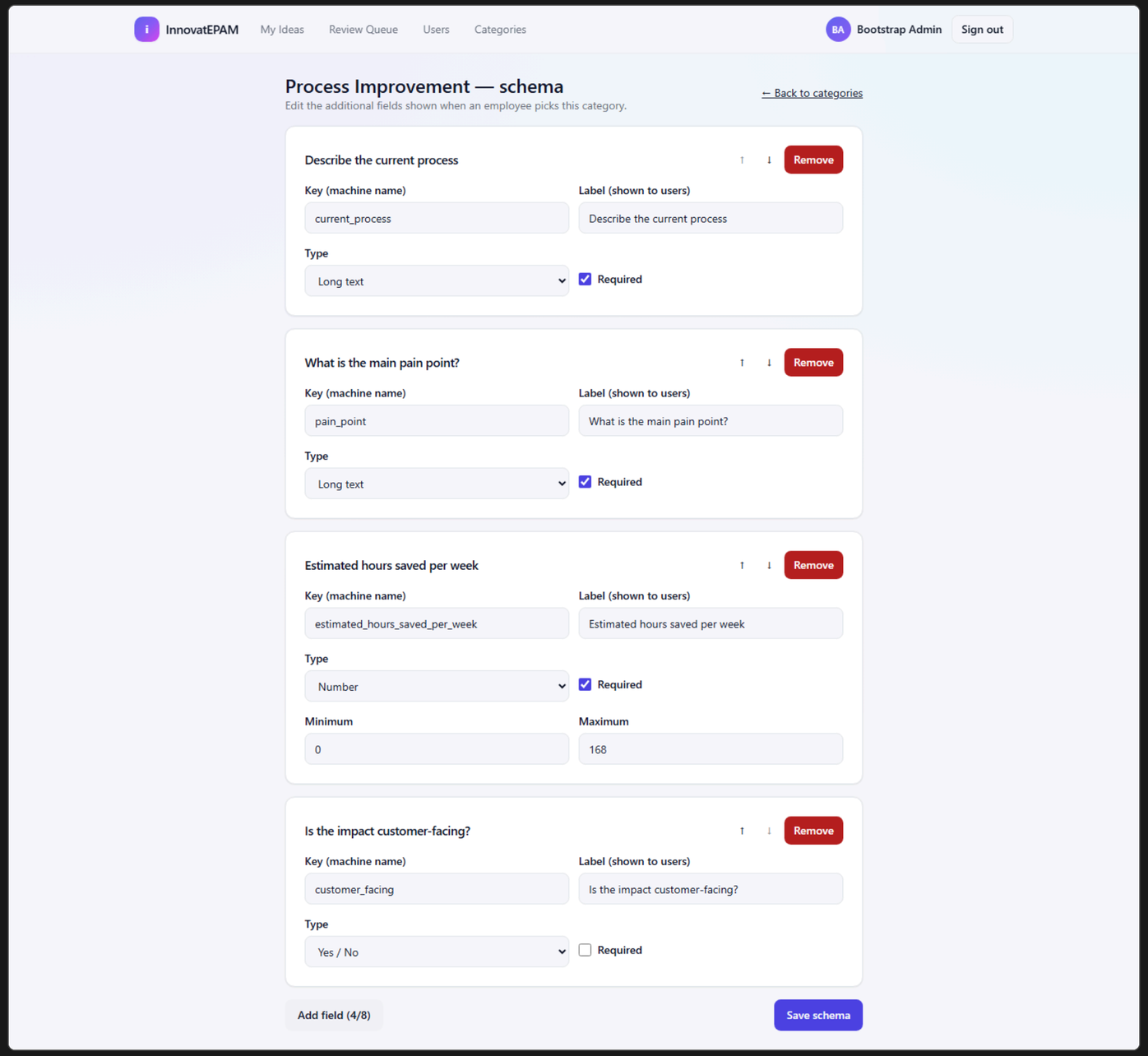
Admin — Categories



`PROPOSED` queue (employee-suggested) → admin approves to `ACTIVE` or rejects. Rejected categories' ideas re-link to the protected **Other**.

screenshots/10-admin-categories.png

Admin — Per-Category Schema Editor



Add / remove / reorder fields. Saved schema becomes the live **source of truth** for new submissions; existing answers keep their snapshot labels, so renames never lose review-time meaning.

screenshots/11-admin-category-schema.png

Tech Stack

Web

- Next.js 14 (App Router, RSC-first)
- React 18 + TypeScript 5 strict
- Tailwind + shadcn/ui (Radix)
- react-hook-form + Zod

Server

- NextAuth v5 + Credentials
- Argon2id passwords
- SQLite (better-sqlite3)
- Drizzle ORM 0.33 + drizzle-kit
- Pino structured logs, rate limiter

Quality

- Vitest unit + integration
- Playwright E2E (chromium / firefox / webkit)
- axe-core accessibility checks
- ESLint + jsx-a11y + Prettier
- Custom `check:error-codes`, `check:ui-tokens`

12 ADRs documented (`specs/001-…/adr/` + `specs/002-…/adr/`) — every non-trivial choice is justified and dated.

By the Numbers

78.8%

Statement coverage

(1506 / 1912)

81.6%

Branch coverage

84%

Function coverage

13

Vitest test files

(4 unit + 9 integration)

4

Playwright E2E specs

(US1–US4 happy paths)

12

ADRs documented

Constitution gate: ≥70% coverage on `src/server/**` + `src/lib/**` — **met**. CI also enforces every `ERROR_CODES` entry to be exercised by ≥1 test, no raw hex/rgb literals in UI, and clean Prettier / ESLint / `tsc --noEmit`.

Architecture Highlights

- **RSC-first** — every protected page does its own `auth()` + `redirect()`; no client trust.
- **Pure-function state machine** in `src/server/idea-state-machine.ts` — same evaluator in API transaction and in UI button gating.
- **Defence-in-depth attachments** — staged on disk, magic-number sniffed, then atomically committed when the idea row is created.
- **Typed error codes** + matching message registry — UI strings never drift from API contracts.
- **Structured security audit log** — every login, role change, transition, attachment.
- **Local-first** — single SQLite file in `data/`; works fully offline.

Spec-Driven Development in Practice

For each phase

- `spec.md` — user stories & acceptance criteria
- `plan.md` — tech approach
- `research.md` — options weighed
- `data-model.md` — tables & constraints
- `contracts/openapi.yaml` — API shape
- `tasks.md` — granular implementation list
- `adr/` — irreversible decisions, dated

Constitution as quality gate

- Coverage thresholds enforced in CI
- UI tokens only — no raw hex / rgb literals
- Every `ERROR_CODE` exercised by ≥1 test
- Drizzle migrations + seed run cleanly
- Strict TS: `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`

Professional Use of AI — Concrete Examples

Where Copilot accelerated me

- Drafting Drizzle schemas + migrations from `data-model.md`
- Generating Zod schemas paired with TS types from OpenAPI
- Scaffolding RSC pages, server actions, shadcn forms
- Producing Playwright specs from acceptance criteria
- Mechanical refactors across the typed error registry

Where I overrode Copilot

- State machine kept as a **pure function**, not inlined in handlers — same evaluator UI + API
- Attachments **staged → committed atomically** with magic-number sniff (security, not the first suggestion)
- Smart-form answers store **label snapshots** so renames don’t rewrite history (ADR-0010)
- Auth gates re-checked server-side on every RSC, not trusted from client
- Every irreversible choice captured in an **ADR**, not just a commit

External Technical Evaluation

Independent code-audit summary (read-only inspection of the repository at `main`).

Stats

- **Codebase:** 179 TS/TSX files in `src` (~13.3k LOC)
- **Tests:** 56 files (~3.8k LOC) — 347/347 green
- **Surface:** 43 React components · 31 API route handlers · 5 Drizzle migrations
- **Spec-driven:** 5 SpecKit phases (MVP → Smart Forms → Listing → Advanced Evaluation → Attachments/History/Notifications) each with ADRs
- **Author:** solo (`ffbilsel`) · **Node:** `>=20 <21`

Architecture

Tech stack verdict

- Next.js 14 App Router · React 18 · TS 5.4 strict (`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`)
- SQLite (`better-sqlite3`) + Drizzle 0.33 · NextAuth v5 (Credentials + DrizzleAdapter, DB sessions, argon2)
- Zod + typed `ERROR_CODES` registry · shadcn/Radix + Tailwind tokens (no hex literals) · Recharts
- `pino`, `rate-limiter-flexible`, `nodemailer`, `file-type`, `diff`
- Vitest unit+integration · Playwright e2e + `@axe-core/playwright` · ESLint + Prettier · custom `check:error-codes` / `check:ui-tokens`
- Dockerfile + docker-compose

Capabilities confirmed

- Service layer in `src/server`: 20+ pure-ish services (state machine, anonymity, drafts, ratings, comments, notifications, email dispatcher, insights, CSV export, attachments, audit history, role-guard, rate-limit)
- Route groups `(admin)`/`(employee)`/`(reviewer)`/`(public)` + `api/` for RBAC isolation
- Pure idea state machine (ADR-0004) · local-first SQLite + on-disk uploads
- RBAC w/ route-group + server guards; smart per-category JSON schemas → runtime Zod + label snapshots
- Submit → review → approve/reject with audit-row diffs; soft-edit window
- Private autosaved evaluator drafts; multi-dim 1–5 ratings with gating + post-decision lock; HTML-escaped comments (5-min edit); per-idea anonymity snapshot
- Shared server-side filter/search/pagination · streaming RFC-4180 CSV export
- Attachments ≤10/idea, ≤10 MB, magic-byte MIME sniff, inline preview (PNG/JPEG/GIF/PDF)
- Unified history timeline + email dispatcher + per-user preferences
- Insights dashboards (trend, approval rate, category dist) — admin vs aggregate-only evaluator views

Assessment (verbatim)

“Mature, disciplined SpecKit-driven solo project. Strong separation of concerns (service layer vs route handlers), strict TS, explicit ADRs per decision, custom static-analysis scripts enforcing design-token + error-code hygiene. Test surface is decent (~29% test-to-src LOC) but skews unit/integration; e2e coverage worth verifying. Auth on NextAuth v5 **beta** is the main external risk; SQLite + local disk caps horizontal scale but matches the ‘small-org local-first’ mandate.”

Thank you

Questions?

Fatih Furkan Bilsel · 12 May 2026

EPAM A201 — Beyond Vibe Coding

Takeaway: AI tools amplify whatever discipline you bring to them.
Spec, ADRs, types and CI are how I made that amplification safe.